

1 Goldwasser-Micali Cryptosystem

A **Group Homomorphic Group Operation** is a group operation over group $\mathbb{G}$ that follows the following property:

$$h(x) \circ h(y) = h(x \circ y)$$

1.1 Setup

This cryptosystem is performed over a finite field F_N where $N = p \cdot q$ for two large prime numbers p, q . p, q are the secret keys and the pk is N .

1.2 Quadratic Residues

An element x is a **Quadratic Residue** mod N if $\exists y \in Z$ such that $y^2 \bmod N \equiv x$. The following facts follow:

Fact 1: $\forall x$, x is a quadratic residue mod N iff x is a quadratic residue mod p AND x is a quadratic residue mod q .

Fact 2: Let p be an odd prime. Then, the value of JACOBI symbol $J_p(x)$ for all x is defined as:

$$J_p(x) = (x^{\frac{p-1}{2}} \bmod p)$$

This is not obvious, but for any number in the group, the only possible values for the Jacobi Symbol is either 1 or -1 for prime order groups.

Fact 3: Let $J_p(x)$ denote the value of Jacobi symbol on x with an odd prime p . Then, x is a quadratic residue mod p iff $J_p(x) = 1$.

Fact 4: If N is a composite of two odd prime numbers p and q then

$$J_N(x) = J_p(x) \cdot J_q(x)$$

The assumption about Quadratic Residuity is that given $N = p \cdot q$ for sufficiently large primes p and q , no PPT algorithm can efficiently determine whether x is a quadratic residue mod N .

1.3 Protocol

The protocol is relatively simple. To encrypt an m , output a randomly chosen quadratic residue mod N for $b = 0$ and a non-residue for $b = 1$. For decryption, the secret keys are used to compute $J_q(c)$ and $J_p(c)$ to determine if the cipher c is a 1 or 0.

There are a bunch more notes about the number theory / math behind the cryptosystem but the main ideas were covered so I am skipping them here.

The punchline is the above cryptosystem has the following property on the ciphertexts of an encryptions $E(y), E(x)$.

$$E(x \oplus y) = E(x) \cdot E(y)$$

This means that if we multiply two group elements that are either QR or QNR, then the resulting element will represent the XOR of the two encrypted elements. More formally:

- A QR $\cdot$ QNR is a QNR
- A QNR $\cdot$ QNR is a QR
- A QR $\cdot$ QR is a QR

2 Private Information Retrieval

Imagine a database D that is N bits long for some very large value N . Now imagine a user u who wants to retrieve the i th bit x_i from the database. However, u does not want the database to learn what bit it requested. A trivial approach to this would be sending the entire database to the user, however, this is very expensive as it requires to send N bits over some channel. The following is a protocol that uses $O(\sqrt{n})$ communication complexity that achieves this retrieval.

2.1 Protocol

This protocol requires an encryption scheme that that is fully homomorphic that follows the following homomorphism:

$$E(x) \cdot E(y) = E(x \oplus y)$$

The protocol is as follows:

1. The database D will split the entire database into $\sqrt{N}$ length blocks so there are $\sqrt{n}$ blocks of length $\sqrt{n}$
2. For each block, U sends two values representing either a 0 or a 1 as $E(0)_j, E(1)_j$, encryptions of both values. However, for all blocks that do not contain the index of the bit U wants, it actually just sends two encryptions of 0.
3. For every bit in the DB, D replaces each bit with $E(b_i)$ sent by the user. Observe that for all bits except for the $\sqrt{N}$ that exist in the block that contains the i th bit, D is replacing each bit with an encryption of 0.
4. Now, D calculates, for the j th bit in each block,

$$E(b_j) \oplus E(b_{j+\sqrt{n}}) \dots \oplus E(b_{j+n-\sqrt{n}})$$

That is, the group operator on all encrypted bits at the j th index of each block. Since all blocks only encrypt 0 except for the one with i , the above protocol will generate that block exactly since it becomes encryption:

$$E(0 + 0 \dots + E(b_{j+k\sqrt{n}}) \dots + 0)$$

5. D sends this back to U , who checks the i th index to get its desired bit.

2.2 PIR $\implies$ Collision Resistant Hashing

The following is a proof that any non-trivial PIR, that is one that has sublinear communication complexity, implies a collision resistant hash. The protocol is as follows:

Protocol: Fix some index i as the PIR query for database x as well as the query for i . Then $h(x)$ is simply the PIR response.

Suppose that this scheme is not secure, meaning that an adversary can find two inputs x, y such that $h(x) = h(y)$. This means that the i th bit must be the same in both x and y and that there is some index j that is different. This immediately gives the adversary some information about the requested index i meaning that the PIR is insecure, a contradiction.

3 Multiparty Computation

Consider two parties, Alice and Bob, who have inputs x, y respectively, that wish to compute some function f without revealing any information beyond $f(x, y)$. It can be accomplished for the following gates as follows:

XOR: Reveal x and y to get $x \oplus y$. This is okay since $f(x, y)$ immediately reveals the inputs to both parties so no extra information is leaked.

AND: This cannot be done without some sort of cryptographic protocol, so we define the notion of a 1 – 2 Oblivious Transfer:

3.1 1-2 OT

This device defined by [Even, Goldreich, Lempel] has two parties, a sender S and receiver R and acts as follows:

1. S provides two bits as input b_0 and b_1 and inputs it into the 1 – 2 OT.
2. R can query the OT one time with s that is either 0 or 1 and receives b_s .

Now, we can design a secure protocol for computing AND as follows:

1. Alice inputs $b_0 = x \wedge 0$ and $b_1 = x \wedge 1$.
2. Bob inputs s as y to get $y \wedge x$ and outputs it to Alice.

3.2 Multi Party XOR/AND computation

Consider n parties who wish to compute the $\oplus$ of their bits $b_1, \dots, b_n$. This can be accomplished if the parties can communicate pairwise with one another. It can be accomplished in the following manner.

1. Player 1 p_1 xors their bits with a completely random bitstring to get $m_1 = x_1 \oplus r$.
2. Each subsequent player calculates their message $m_i = x_i \oplus m_{i-1}$ for $i = 2 \dots n$.
3. Player n sends m_n to p_1 who calculates $r \oplus m_n = \bigoplus_{i=1}^n x_i$ and outputs the result to all other parties.

4 Secure Computation

We discuss a secure 2-Party Computation that can compute a circuit consisting of only XOR and AND gates. To do this, there is a notion of secret sharing that inductively implies throughout the circuit from input to output. We also assume that the circuit outputs either 0/1, meaning it has one output wire at the top level.

4.1 Secret Sharing

Secret sharing involves splitting a secret value S into shares $s_1, \dots, s_n$ distributed among n players, given a certain number of shares, say t , one can recover the secret S .

4.2 Setup

We consider the two party case where parties Alice A and Bob B have inputs $X = x_1, \dots, x_n$ and $Y = y_1, \dots, y_n$ respectively. They need to split their inputs into shares such that using those shares, they can compute the corresponding outputs to XOR and AND gates produce corresponding shares to the output of the gate. We split up A 's input into shares by selecting a random a_i for each bit x_i and creating share $a'_i = x_i \oplus a_i$. Now A 's share is a_i and B 's share of x_i is a'_i . This is also done for Y to create shares b_i and b'_i . Note that A has shares that we will call a_1, b_1 and B has shares a_2, b_2 and that $a_1 \oplus a_2 = x$ and $b_1 \oplus b_2 = y$.

4.3 XOR Gates

To simulate an XOR gate, we can simply create shares for A and B as:

$$a_s = a_1 \oplus b_1$$

$$b_s = a_2 \oplus b_2$$

This works since we are trying to compute the value of $a \oplus b$ which can be represented as $a_1 \oplus a_2 \oplus b_1 \oplus b_2$. This value is split into two shares above and therefore XOR gates can be simulated.

4.4 AND gates

To compute AND gates, we need to compute $(a_1 \oplus a_2)(b_1 \oplus b_2)$. To accomplish this, we distribute to get:

$$(a_1 \oplus a_2)(b_1 \oplus b_2) = a_1b_1 \oplus a_1b_2 \oplus a_2b_1 \oplus a_2b_2$$

Recall that A has shares a_1, b_1 and B has shares a_2, b_2 so the first and last terms can be calculated. They calculate terms a_1b_2 and a_2b_1 in the following manner:

1. A uses 1-2 OT to send $s_0 = a_10 \oplus r$ and $s_1 = a_11 \oplus r$ to B who inputs $s = b_2$ to obtain $a_1b_2 \oplus r$
2. The same is done with B to compute $a_2b_1 \oplus r'$ which is given to A
3. A generates share $r \oplus a_1b_1 \oplus a_2b_1 \oplus r'$
4. B generates share $r' \oplus a_2b_2 \oplus a_1b_2 \oplus r$

The above shares clearly result in shares for AB demonstrating the correctness of the AND gate.